WEEK 09

Functions, filters and tests

Python foundations

Functions, filters and tests

Predicates, result lists and independent tests

A predicate asks a question

Its Boolean result decides whether an item belongs.

```
def positive(n): return n > 0
print(positive(0))  # False
```

Filtering builds a result

Keep matching values without changing the source list.

```
values = [-1, 0, 2]
kept = [n for n in values if n > 0]  # [2]
```

Tests need independent expectations

A boundary case distinguishes > from >=.

```
def positive(n): return n > 0
assert positive(0) is False
```

A chore list answers a precise question

Question: which enabled chores are unfinished?

Chore	enabled	finished	Select?
A	**True**	**False**	**Yes**
B	True	True	No
C	False	False	No

Result: ["A"]

```
row["enabled"] and not row["finished"]
```

The result is a new list; the original records stay unchanged.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
def feature(inputs):	Give the change a boundary	Make behavior callable
[x for x in rows if ...]	Filter a collection	Choose the intended records
assert actual == expected	Compare against a prediction	Expose a wrong implementation
check.assertEqual(a, b)	Assert with unittest	Reuse checks in a test module

Python Basics · sections 07, 10, 15, 17 · use only when you need a lookup.

Make one predicate from one record

```
def needs_work(row):
    return row["enabled"] and not row["finished"]

row = {"enabled": True, "finished": False, "published": False}
print(needs_work(row))
assert needs_work(row) is True
```

1. A predicate returns a Boolean.

Build a new result instead of changing input

```
numbers = [1, 2, 3]
selected = []
for number in numbers:
    if number >= 2:
        selected.append(number)
print(selected)
print(numbers)
assert numbers == [1, 2, 3]
```

1. A new result list leaves the input unchanged.

Keep an independent before snapshot

```
import copy
records = [{"points": 1}]
alias = records
before = copy.deepcopy(records)
records[0]["points"] = 9
print(alias[0]["points"], before[0]["points"])
assert before == [{"points": 1}]
```

1. A second name such as alias still refers to the original object.

Check a boundary and filter a small collection

```
def at_least(value, minimum):
    return value >= minimum
for value in (4, 5, 6):
    print(value, at_least(value, 5))
records = [("A", False), ("B", True), ("C", False)]
pending = [name for name, finished in records if not finished]
print(pending)
```

1. Expected: 4 False; 5 True; 6 True; ['A', 'C'].
2. A predicate decides which values to keep.
3. Replace >= with >: which check should fail?

Find a boundary bug in a function

```
01 def before_label(portions, reviewed):
02     if not reviewed:
03         return "hold"
04     return "team" if portions > 20 else "standard"
05
06 print(before_label(20, True))  # Wrong: standard.
07 print(before_label(20, False)) # Correct: hold.
```

1. Trace the early return first.
2. Compare > with >= at equality.
3. Keep an independent expected result.

An exact-boundary input can expose a one-character bug.

Table-driven checks for a small patch

```
01 def pickup_label(portions, reviewed):
02     if not reviewed:
03         return "hold"
04     return "team" if portions >= 20 else "standard"
05
06 cases = [(19, True, "standard"), (20, True, "team"),
07          (21, True, "team"), (20, False, "hold")]
08 for portions, reviewed, expected in cases:
09     assert pickup_label(portions, reviewed) == expected
```

1. Each tuple supplies inputs and expectation.
2. The loop runs one check per tuple.
3. The first failing assert stops execution.

A table of cases checks the patch and preserved behavior.

List comprehensions: choose the right predicate

```
rows = [("D10", False, False),
        ("D11", True, False), ("D12", True, True)]
pending = [key for key, collected, delivered in rows
           if not collected]
wrong = [key for key, collected, delivered in rows
         if not delivered]
assert pending == ["D10"]
assert wrong == ["D10", "D11"]
print(pending, wrong)
```

1. Unpack each tuple into three names.
2. The if clause filters the result list.
3. Find the row that distinguishes the filters.

Changing a filter condition can change which items are included.

Import reusable code without running startup

```
01 from pickup_app import planned_portions
02
03 print(planned_portions(3, 3))
04 assert planned_portions(3, 3) == 9
05 # This import does not reserve or deliver anything.
```

1. Use the documented module location.
2. Tests can call a function directly.
3. Keep input/output work in the entry point.

Reusable code can be imported without starting the application.

unittest: assert a returned value

```
01 import unittest
02
03 def label(portions, reviewed):
04     if not reviewed:
05         return "hold"
06     return "team" if portions >= 20 else "standard"
07
08 check = unittest.TestCase()
09 check.assertEqual(label(20, True), "team")
10 check.assertEqual(label(20, False), "hold")
```

1. Import the actual function in your project tests
2. Retain independent expected labels
3. Add negative, failure and state-preservation cases

A test must execute an independent expectation, not remain skipped.

Hands-on practice

Week_09.ipynb · Session 2

Python practice

P1–P2 · Core

Two small programs with approach hints.

P3 · Core

A complete program with normal and edge-case tests.

P4–P5 · Optional

An application and a harder challenge.

P1–P3: your code, expected results and an explanation. P4–P5 are optional.

Project studio

Student-designed feature

The project need

1

Need

Choose one small problem a volunteer actually faces.

2

Operation

Implement your approved small change and check it with another learner.

3

Result

A volunteer can try the changed feature and report what happened.

Example: make pending pickups easier to find.

Turn the need into an acceptance experiment

1

Need

Identify the coordinator decision and present difficulty.

2

Contract

Specify inputs, permitted action and observable output.

3

Cases

Predict a normal case and a distinguishing boundary or failure.

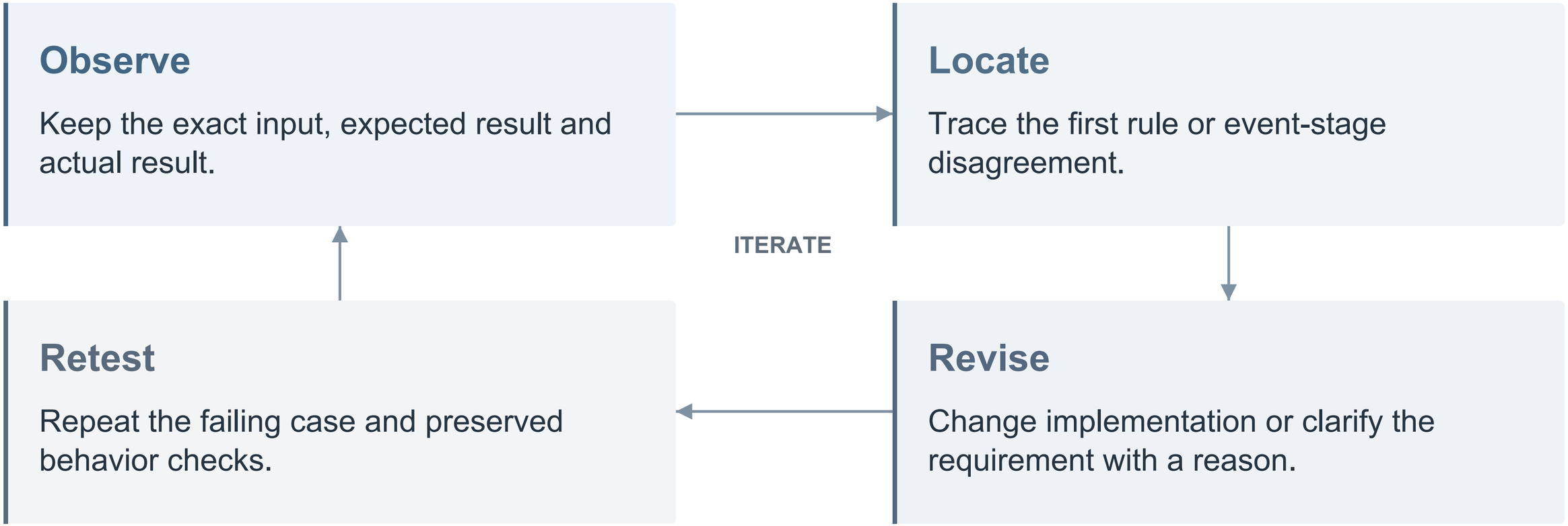
4

Evidence

Run independently and compare actual records and outputs.

Expected behavior comes from the need, not from the program output.

Use a failing observation to guide a revision



Find the first disagreement before changing the expected result.

Project task

Supplied starting point

Baseline, fixtures and sample tests are supplied; they are the starting point, not your finished feature.

You implement / do

State the user need; implement the feature; write two acceptance cases plus an unaffected regression and a boundary/failure case.

Check the evidence

Run your actual feature entry point. Record a real peer finding, revise and retest. For a pending-collection view, distinguish collection from delivery.

Use the supplied baseline; explain a design decision and a limitation.

Project checks

Step	What to show
Implementation	Implement your approved feature and run actual tests.
Checks	Two acceptance cases; regression; failure; peer retest.

Use the supplied baseline; explain a design decision and a limitation.

Further reading in English

1. ## Python unittest documentation

   https://docs.python.org/3/library/unittest.html

2. ## Google Engineering Practices: How to do a code review

   https://google.github.io/eng-practices/review/reviewer/

Read one relevant section, then bring an example and a question.